Utilisation du programme de simulation d'une machine virtuelle,
Document adressé aux développeurs

Travail effectué par : Thomas Gautier et Baptiste Bigot

I. Introduction au problème

Le but de ce programme est de simuler une machine fictive composée d'une mémoire et d'un microprocesseur. Il est écrit en C et comporte ___ fichiers. Pour l'utilisation de la machine voici le document vous fournissant l'aide nécessaire.

Voici comment se compose ce document :

1. Introduction
2. Structure et arborescence du programme
3. Compilation
   a. Structure de la compilation et stratégie adopté 1/2
   b. Structure adoptée pour la compilation et l'encodage 2/2
   c. Explication des fichiers assembleur.c et codage.c
   d. Utilisation de l'IA sur cette partie
   e. Faille et piste d'amélioration
4. Micro-processeur
   a) La présentation
   b) Les failles et améliorations potentielles

II. Structure et arborescence du programme

Le programme est divisé en deux parties distinctes : le compilateur (l'assembleur) et le processeur (la machine virtuelle). Le programme est envoyé dans le compilateur par la commande utilisateur dans le terminal. L'architecture global repose principalement sur un fonctionnement en chaine :

1. L'utilisateur renvoie le fichier sous format assembleur dans un document .txt (ex : test.txt)
2. Le document est récupéré par le fichier main et l'envoi à l'assembleur qui vérifie si le document est valide, c'est-à-dire aucune erreur. Ensuite il le traduit en hexadécimal dans un fichier appelé hexa.txt.
3. Enfin, le programme est envoyé au CPU charge les instructions dans sa mémoire simulée et les exécute une par une et affiche le résultat dans le terminal.

III. La compilation

A. Structure de la compilation et stratégie adopté

Pour transformer le fichier texte en code machine, nous pensions que c'était un jeu d'enfant nécessitant seulement de traduire les codes opérateurs en hexadécimal et hop la compilation était terminée. Mais après une petite réflexion nous nous sommes rendu compte que nous devions faire face à un problème majeur : les sauts conditionnels et les étiquettes. Le problème principal est le référencement des étiquettes qui ne sont pas encore définies (Ex : JMP Fin, alors que Fin se trouve à la fin du fichier). Pour contrer ce problème, on a d'abord imaginé lire le fichier deux fois, mais les accès disque sont lents. On a donc fait mieux ! En ne le lisant qu'une seule fois pour tout mettre en mémoire, puis on adopte une architecture à deux passes sur ces données :

1. Lors de la lecture du fichier, le programme nettoie le code immédiatement. Il retire les commentaires, sépare les mots et vérifie la syntaxe (si ADD a bien 3 arguments, etc.). Tout est stocké dans un grand tableau de structures en mémoire. C'est seulement après qu'on parcourt ce tableau pour repérer les mots-clés terminant par « : ». On stock le nom de l'étiquette et on calcul sa future adresse (via son index * 4 octets) dans une structure appelée Table des symboles.
2. Ensuite, la compilation finale s'effectue. Le programme relit le tableau d'instructions une seconde fois. Comme la syntaxe est déjà validée, il se concentre sur les arguments manquants. Par exemple, s'il tombe sur JMP boucle, il voit que "boucle" n'est ni un registre ni un nombre. Il regarde alors dans la Table des symboles remplie à l'étape précédente. S'il trouve le nom, il remplace le texte par l'adresse mémoire correspondante. Enfin, il renvoie la structure complète prête à être encodée en hexadécimal.

B. Structure adoptée pour la compilation et le codage en hexadécimal

Une fois l'analyse syntaxique et sémantique validée par l'assembleur, le programme dispose d'une liste de structures contenant les instructions "propre'' qu'il va traduire en hexadécimal, format seul compréhensible par le processeur. Nous avons rigoureusement suivi le format d'instruction sur 32 bits, donné en cours, où chaque plage de bits a une fonction précise : les bits 31 à 27 définissent le code opérateur, suivis du registre destinataire (26-22) et de la source (21-17). Le bit 16 agit comme un indicateur de type (immédiat ou registre) pour interpréter les bits 15 à 0, qui contiennent la valeur ou l'adresse mémoire (correspondant aux 64 Ko adressables). Techniquement,

cette phase ne réalise pas de calculs arithmétiques mais de la manipulation binaire pure. Le programme construit l'instruction finale en assemblant les différents champs via des opérations de décalage logique. Par exemple, pour positionner l'opcode en tête du mot de 32 bits, nous appliquons à sa valeur un décalage à gauche de 27 rangs (opcode << 27).

C. Explication des fichiers assembleur.c et codage.c

a) assembleur.c

D'abord, la stratégie de compilation repose sur une architecture très cadré, où le fichier assembleur.c réalise l'essentiel du travail d'analyse textuelle et logique. La logique est orchestrée par la fonction principale assembler, qui agit comme le point d'entrée du processus de traduction. Son rôle premier est d'ouvrir le fichier source et d'initier une boucle de lecture qui traite le code ligne par ligne. Plutôt que de traduire le code à la volée. Elle nettoie les problèmes de formatage, appelle la logique d'analyse pour chaque ligne et stocke le résultat sous forme de structures dans un tableau en mémoire. Elle prend en argument un tableau d'Instruction, correspondant aux différentes lignes du programme assembleur qui seront remplis, un pointeur sur le nombre d'instruction du programme, et le nom du fichier. Elle remplit le tableau et le nombre d'instruction, pour la suite du programme.

L'analyse lexicale pure est déléguée à la fonction lit_ligne. Ce composant est chargé de disséquer chaque chaîne de caractères brute pour en extraire les informations utiles. Il commence par repérer et supprimer les commentaires pour ne conserver que le code exécutable, puis détecte la présence éventuelle d'étiquettes définies par des deux-points. Une fois la structure de la ligne clarifiée, la fonction découpe le reste du texte en utilisant les espaces, les parenthèses et les virgules comme séparateurs pour isoler le code opérateur et ses arguments, qu'elle classe ensuite comme registres ou valeurs immédiates.

Avant toute traduction finale, la validité des données extraites est assurée par la fonction verif_tout. Cette étape est essentielle pour la stabilité du programme, car elle analyse chaque instruction dès sa lecture pour effectuer des tests sémantiques stricts. Elle s'assure notamment de la cohérence des opérandes : elle vérifie que le nombre d'arguments fournis respecte la signature de l'instruction (ex: 3 arguments pour un calcul), que les types sont respectés (pas de valeur immédiate pour une destination) et que les registres sollicités existent bien dans l'architecture du processeur (R0 à R31).

Enfin, la résolution des références symboliques est traitée par la fonction traiter_labels. Cette fonction opère en parcourant le tableau d'instructions en deux

temps. Une première passe permet de cartographier le programme en associant chaque étiquette déclarée à son adresse mémoire réelle (calculée par index * 4 octets). Une seconde passe relit ensuite l'ensemble des instructions pour repérer les arguments qui ne sont ni des registres, ni des valeurs immédiates, ni des adresses mémoire explicites ; lorsqu'une correspondance avec un label connu est trouvée, le nom textuel est remplacé par sa valeur numérique immédiate (ex: #12), rendant le code prêt pour l'encodage hexadécimal.

```
// Extrait de la fonction traiter_labels() dans assembleur.c
// etapes 2 : Remplacement des symboles par les adresses mémoires
for (int i = 0; i < n; i++) {
    for (int j = 0; j < prog[i].nb_args; j++) {
        char *arg = prog[i].args[j];
        // Si l'argument n'est ni un registre, ni une valeur immédiate, ni une adresse mémoire...
        // ...alors c'est une Étiquette à résoudre
        if (!est_reg(arg) && !est_imm(arg) && !est_mem(arg)) {

            int trouve = 0;
            //recherche linéaire dans la table des symboles
            for (int k = 0; k < nb_lab; k++) {
                if (strcmp(arg, tab_labels[k].nom) == 0) {

                    //on écrase le texte du label par son adresse numérique
                    //par exemple "Boucle" devient "#12"
                    sprintf(prog[i].args[j], "#%d", tab_labels[k].adr);

                    tab_labels[k].use = 1; // On marque le label comme utilisé (pour les Warnings)
                    trouve = 1;
                    break;
                }
            }
            // Gestion d'erreur : Si le label n'est pas dans la table
            if (!trouve) {
                printf("Erreur critique : Label '%s' indéfini\n", arg);
                err++;
            }
        }
    }
}
```

Voici une capture d'écran de la deuxième étape de la fonction traiter_label. On peut peut-être rajouter si cela est nécessaire que sprintf permet d'écrire, non pas dans la console comme son cousin printf, mais dans un buffer, son utilisation est peut-être dangereuse car il peux y avoir des risques de dépassement.

b) codage.c

Le fichier codage.c joue un rôle clé dans la compilation d'un fichier, elle fonctionne de pair à celle d'assembleur.c. Sa mission est strictement déterministe : convertir les structures de données validées en une suite de nombres hexadécimaux exécutables par le processeur. Le code repose sur un ensemble de définitions constantes (des macros en fait) qui associent chaque mnémonique textuel, tel que ADD ou LDB, à un code numérique unique.

Le processus de conversion s'appuie sur plusieurs fonctions utilitaires spécialisées. La fonction trouver assure la correspondance entre le texte et le code opérationnel

numérique. Elle intègre une sécurité défensive nettoyant les chaînes de caractères pour éliminer les retours à la ligne résiduels, garantissant qu'une instruction ADD soit reconnue même si le fichier source contient des formatages exotiques. En parallèle, les fonctions registre et valeur se chargent d'extraire les opérandes. La fonction registre est capable d'analyser la syntaxe spécifique pour gérer les parenthèses indiquant un accès mémoire, comme (R1), tandis que la fonction valeur utilise strtol pour interpréter indifféremment des nombres décimaux ou hexadécimaux préfixés.

Le cœur logique du programme réside dans la fonction encoder. C'est ici que l'instruction prend sa forme binaire définitive de 32 bits. La fonction commence par identifier la catégorie de l'opération (mathématique, mémoire, saut ou entrée/sortie) pour déterminer comment répartir les arguments dans les champs destination, source et valeur immédiate. Une sécurité critique a été implémentée à ce niveau : si une instruction est inconnue, la fonction génère un code "zéro" plutôt qu'une instruction HLT par défaut, évitant ainsi un arrêt involontaire du processeur.

L'assemblage final du mot machine est réalisé par des opérations bit à bit. Le programme initialise un entier nul puis y insère chaque composant à sa place précise par décalage logique vers la gauche (<<). L'opcode est placé sur les 5 bits de poids fort, suivi des identifiants de registres, du drapeau de type et enfin de la valeur immédiate ou de l'adresse sur les 16 bits de poids faible. Une fois cet entier construit, la fonction ecrire prend le relais pour générer le fichier de sortie hexa.txt. Elle parcourt l'ensemble du programme traduit et écrit chaque instruction sous la forme d'une chaîne hexadécimale à 8 caractères, format standardisé prêt à être chargé par la machine virtuelle.

```c
// Extrait de la fonction encoder() dans codage.c
unsigned int encoder(Instruction inst) {
    // 1. Récupération de l'ID numérique de l'instruction
    int op = trouver(inst.opcode);
    if (op == -1) return 0; // Sécurité

    int dest = 0, src = 0, flag = 0, val = 0;

    // 2. Exemple de préparation des données : Cas Mathématiques
    if ((op >= 0 && op <= 7) || op == OP_RND) {
        dest = registre(inst.args[0]);
        src  = registre(inst.args[1]);

        // Détection : est ce une valeur immédiate (#10) ou un registre (R2) ?
        if (inst.args[2][0] == '#') {
            flag = 1;
            val = valeur(inst.args[2]);
        } else {
            flag = 0;
            val = registre(inst.args[2]);
        }
    }
    // ... (Autres cas traités : Mémoire, Sauts, E/S) ...
    // 3. Assemblage du mot de 32 bits final (Bitwise Assembly)
    unsigned int res = 0;
    // On utilise le OU logique (|) pour fusionner les bits
    // On utilise le décalage (<<) pour placer les champs au bon endroit
    // On utilise le ET (&) pour masquer et éviter les débordements
    res |= (op   & 0x1F) << 27;  // Opcode (5 bits) -> Bits 31-27
    res |= (dest & 0x1F) << 22;  // Destination (5 bits) -> Bits 26-22
    res |= (src  & 0x1F) << 17;  // Source (5 bits) -> Bits 21-17
    res |= (flag & 0x1)  << 16;  // Flag Type (1 bit) -> Bit 16
    res |= (val  & 0xFFFF);      // Valeur ou Adresse (16 bits) -> Bits 15-0

    return res;
}
```

Voici un extrait de la fonction encoder, qui permet de convertir une instruction en un code hexadécimal, elle doit être hyper précise car une erreur de décalage ou de mauvaise écriture peuvent créer de nombreux problèmes dans le code. Elle retourne un unsigned int car c'est un code machine il n'est pas négatif.

D. L'utilisation de l'IA (pour la compilation et l'exécution)

Il serait tentant d'affirmer que ce projet a été réalisé intégralement à la main, armé seulement d'un éditeur de texte et de la documentation du langage C. Cependant, il faut vivre avec son temps : l'intelligence artificielle a joué un petit rôle de support dans cette aventure. Affirmer l'inverse manquerait d'honnêteté.

Nous avons utilisé l'IA non pas pour concevoir l'architecture logique du processeur, qui reste le fruit exclusif de notre réflexion, mais comme un outil d'accélération et de débogage. Pour être transparents : la gestion manuelle de la mémoire en C a tendance à générer des erreurs frustrantes (« Segmentation Faults »). Dans ce contexte, l'IA a servi de 'correcteur', capable de repérer une erreur de pointeur ou un décalage de bit (<<) hasardeux plus rapidement que l'œil humain fatigué.

Elle a également été sollicitée pour expliciter ou optimiser la clarté de certaines fonctions, facilitant ainsi la compréhension mutuelle du code entre partenaires. Ces suggestions ont évidemment été utilisées avec modération et **toujours retravaillées**.

Enfin, nous avons utilisé l'outil pour générer des fichiers assembleurs de test complexes, nous permettant de valider les limites de notre programme efficacement.

Enfin, la dernière utilisation de l'ordinateur a été pour générer des fichiers assembleurs afin de tester les limites de notre programme. Cela a été grandement utile car nous savions exactement ce qu'il y avait a retravaillé et ce qui marchait.

Donc, non l'IA n'a pas écrit le projet à notre place, bien loin de là, mais elle a considérablement réduit la consommation d'aspirine nécessaire à sa réalisation...

Enfin, par souci d'intégrité académique, nous tenons à préciser que nous nous sommes inspirés de codes existants sur GitHub pour la conception de la fonction lit_ligne dont la logique s'inspire du projet *LC-3 VM*. Le reste du code est une production originale 'fait maison'.

E. Failles et pistes d'amélioration

Bien que, notre simulateur remplisse son contrat et parvient à exécuter des programmes, il est loin d'être infaillible. Le développement nous a appris que l'IA ne remplace pas la logique de conception, et plusieurs limitations techniques subsistent. Nous sommes loin de l'ère où l'intelligence artificielle remplacera l'humain. Nous vous proposons ici, des pistes d'améliorations ou de perfectionnement des différentes failles présente dans le code :

- La fragilité de lecture du code : évidemment, pour la fonction lit_ligne, qui découpe le texte, nous avons choisi de ne pas réinventer la roue, en nous inspirant du mécanisme "tokenisation'' de LC-3 VM sur le découpage. Cependant, cette méthode a ses limites et notre code est assez capricieux sur la syntaxe. Un espace oublié après une virgule ou un caractère invisible traînant en fin de ligne peut parfois suffire à embrouiller l'assembleur.
- Nous avons fait le choix de charger tout le programme dans un tableau en mémoire (Instruction *prog). C'est simple et efficace pour nos exercices, mais ce n'est pas viable à grande échelle. Sur un très gros projet, cette méthode saturerait la mémoire vive. Une approche plus professionnelle aurait demandé une lecture en flux continu, une complexité que nous avons choisi d'éviter pour ce projet, mais qui serait la première étape d'une potentielle amélioration.
- Enfin, notre machine virtuelle manque de protections mémoire. Comme nous l'avons constaté lors des tests, il est possible pour un programme d'écrire par-dessus ses propres instructions. Si c'est amusant pour comprendre les failles de sécurité, c'est un problème majeur pour un vrai système.

IV. Le microprocesseur

1) Présentation

Le fichier cpu.c constitue le centre du projet car c'est lui qui donne vie au code hexadécimal généré par l'assembleur. Contrairement à ce dernier qui agit comme un traducteur statique, ce module simule le comportement dynamique d'un processeur physique en implémentant le cycle Fetch-Decode-Execute. L'architecture repose sur la simulation des composants matériels via des structures de données en C, notamment un tableau d'octets unsigned char simulant une RAM de 64 Ko adressable octet par octet, et un tableau d'entiers représentant la banque de 32 registres généraux. Le système s'appuie également sur une variable essentielle, le Program Counter (PC), qui retient l'adresse de l'instruction en cours, ainsi que sur trois variables globales mémorisant l'état du processeur : Zéro (Z), Retenue (C) et Négatif (N). Une fonction d'initialisation s'assure de remettre tous ces composants à zéro au lancement pour garantir un état stable.

La mécanique d'exécution est orchestrée par une fonction principale qui mime l'horloge du processeur via une boucle. Par sécurité, cette boucle intègre un mécanisme limitant l'exécution à 50 000 cycles afin d'éviter tout blocage indéfini. La première étape de chaque itération, le "Fetch", consiste à récupérer l'instruction pointée par le PC. Comme notre mémoire est stockée par octets de 8 bits mais que les instructions font 32 bits, le code doit lire quatre cases mémoire consécutives. Nous utilisons des décalages de bits de 24, 16 et 8 positions combinés à des opérateurs logiques OR pour fusionner ces octets et reconstruire l'instruction complète dans une variable temporaire.

Une fois l'instruction reconstituée, la phase de décodage entre en jeu pour la fragmenter. Des masques binaires et des décalages inversés permettent d'isoler l'Opcode sur les 5 bits de poids fort ainsi que les indices des registres cibles. Une logique spécifique gère le second opérande en vérifiant le 16ème bit de l'instruction : s'il est à zéro, la machine lit la valeur dans un registre, mais s'il est à un, elle extrait les 16 derniers bits comme une valeur immédiate. Un point important de cette étape est l'application d'une extension de signe, qui permet que les nombres négatifs codés sur 16 bits conservent leur valeur négative une fois convertis en entiers 32 bits pour les calculs.

L'exécution proprement dite est déléguée à un vaste sélecteur switch qui aiguille le flux vers l'opération demandée. Pour la partie arithmétique, nous avons mis en place une astuce d'implémentation pour gérer les drapeaux : les calculs sont effectués dans une variable temporaire de type long long sur 64 bits. Cette précaution permet de

détecter si le résultat dépasse la capacité d'un entier standard de 32 bits sans perdre l'information. C'est grâce à cette comparaison que la fonction de mise à jour des états peut lever le drapeau de retenue (Carry) si nécessaire, tout en actualisant les drapeaux Zéro et Négatif selon le résultat final. Enfin, pour les instructions d'accès mémoire, l'adresse cible est calculée en additionnant un registre et une valeur de décalage, avec des tests de sécurité systématiques qui vérifient que l'adresse ne sort pas des bornes de la RAM pour éviter tout arrêt critique du simulateur.

```
// Extrait de cpu.c : Boucle principale

// 1.Reconstitution de l'instruction (32 bits) a partir de 4 octets
// On decale chaque octet vers la gauche pour les remettre dans l'ordre
unsigned int instr = (memoire[pc] << 24) | (memoire[pc+1] << 16) |
                     (memoire[pc+2] << 8)  | memoire[pc+3];
pc += 4; // On avance a l'instruction suivante
// 2. décodage On isole l'Opcode (les 5 premiers bits) avec un masque
int opcode = (instr >> 27) & 0x1F;
int rn     = (instr >> 17) & 0x1F; // Registre source
// 3. execution : On lance la bonne operation
switch (opcode) {
    case 3: // ADD
        // Calcul avec protection anti-overflow (long long)
        res = (long long)reg[rn] + op2;
        break;
    // ... autres instructions ...
}
```

Afin de concrétiser cette explication théorique, voici l'extrait central de la boucle d'exécution (de la fonction executer). Ce passage montre ce qu'effectue le code de la réception a l'exécution d'instructions. Evidemment c'est minimaliste.

2) Piste d'amélioration

Bien que notre processeur soit fonctionnel et capable d'exécuter des algorithmes complexes, il reste relativement simple et possède certaines limites structurelles. La première est l'absence de gestion native des nombres à virgule flottante : notre programme est purement centré sur les entiers. L'ajout des nombres flottant via des instructions dédiées serait une évolution majeure.

La seconde limitation notable est l'absence de Pile d'appel et de registre associé. Actuellement, nous ne pouvons effectuer que des sauts simples, ce qui rend impossible l'implémentation de vraies fonctions.

Enfin, la gestion des entrées-sorties reste bloquante (le processeur attend l'utilisateur). Une architecture plus poussée intègrerait un système d'interruptions pour permettre au processeur de continuer à travailler pendant qu'il attend une saisie.